Advanced Computer Systems (CIS 570)

MIPS

(RISC Example)

Instruction Set Architecture

Slides are adopted from:

- Patterson and Hennessy, Computer Architecture a Quantitative Approach (Fifth edition)
- Notes by Dr. O. Mutlu, CMU
- Patterson and Hennessy, Computer Organization and Design: The Hardware/Software Interface, Fourth Edition
- Notes by Dr. M. Geiger, UMA.
- Vincent Heuring and Harry Jordan, Computer Systems design and Architecture, Second Edition

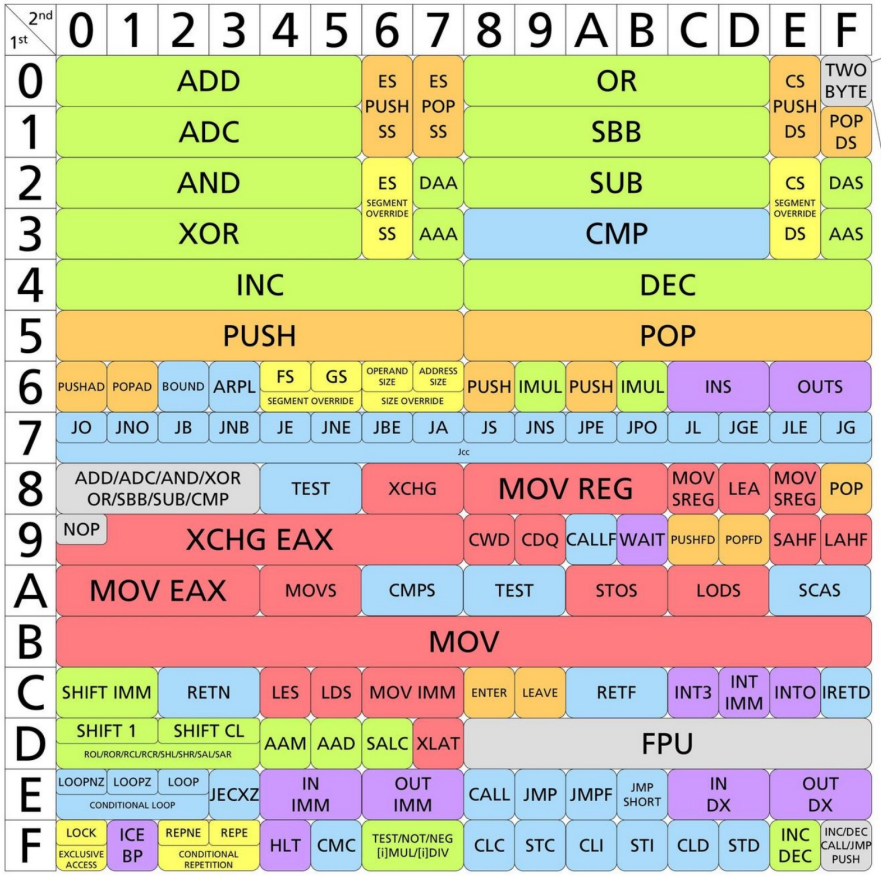
Source:

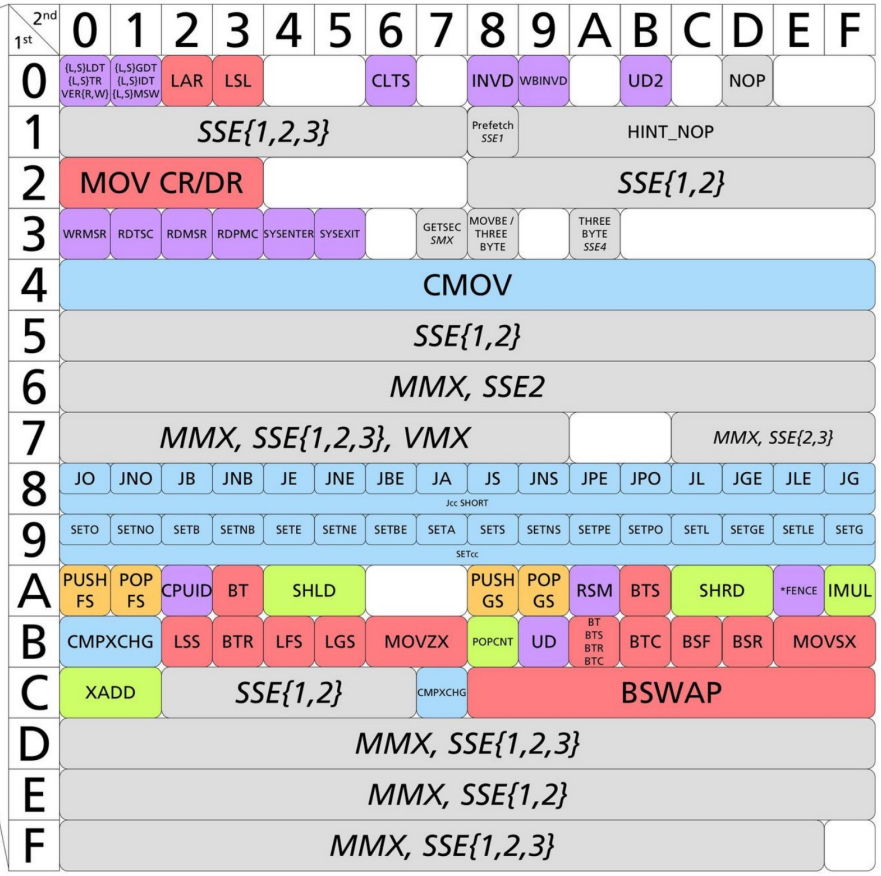
- Chapter 2 from HPK's Computer Organization and Design: The Hardware/Software Interface

X86 instruction Set Reference

Advanced Computer Systems

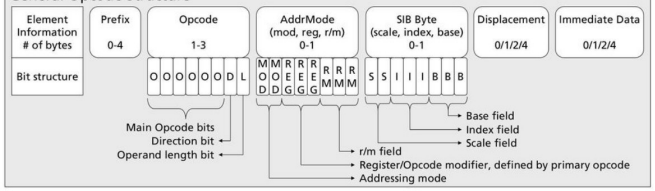
x86 Opcode Structure and Instruction Overview





Arithmetic & Logic
Memory
Stack
Control Flow & Conditional
Prefix
System & I/O
No Operation (NOP) / Multiple Instructions / Extended Instruction Set

General Opcode Structure



Addressing Modes

mod	00		01		10		11
r/m	16bit	32bit	16bit	32bit	16bit	32bit	r/m // REG
000	[BX+SI]	[EAX]	[BX+SI]+disp8	[EAX]+disp8	[BX+SI]+disp16	[EAX]+disp32	AL / AX / EAX
001	[BX+DI]	[ECX]	[BX+DI]+disp8	[ECX]+disp8	[BX+DI]+disp16	[ECX]+disp32	CL / CX / ECX
010	[BP+SI]	[EDX]	[BP+SI]+disp8	[EDX]+disp8	[BP+SI]+disp16	[EDX]+disp32	DL / DX / EDX
011	[BP+DI]	[EBX]	[BP+DI]+disp8	[EBX]+disp8	[BP+DI]+disp16	[EBX]+disp32	BL / BX / EBX
100	[SI]	SIB	[SI]+disp8	SIB+disp8	[SI]+disp16	SIB+disp32	AH / SP / ESP
101	[DI]	disp32	[DI]+disp8	[EBP]+disp8	[DI]+disp16	[EBP]+disp32	CH / BP / EBP
110	disp16	[ESI]	[BP]+disp8	[ESI]+disp8	[BP]+disp16	[ESI]+disp32	DH / SI / ESI
111	[BX]	[EDI]	[BX]+disp8	[EDI]+disp8	[BX]+disp16	[EDI]+disp32	BH / DI / EDI

SIB Byte Structure

encoding	scale (2bit)	Index (3bit)	Base (3bit)
000	2^0=1	[EAX]	EAX
001	2^1=2	[ECX]	ECX
010	2^2=4	[EDX]	EDX
011	2^3=8	[EBX]	EBX
100	--	none	ESP
101	--	[EBP]	disp32 / disp8+ [EBP] / disp32 + [EBP]
110	--	[ESI]	ESI
111	--	[EDI]	EDI
SIB value = index * scale + base			

v1.0 – 30.08.2011
Contact: Daniel Plohmann – +49 228 73 54 228 – daniel.plohmann@fkie.fraunhofer.de

Source: Intel x86 Instruction Set Reference
Opcode table presentation inspired by work of Ange Albertini

MIPS instruction formats

Type	-31-			format (bits)		-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

MIPS instruction formats

A fast instruction is easier to decode

Type	-31-	format (bits)				-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

An easy to decode Inst is simple one

Simplicity favors regularity

MIPS: A "Typical" RISC ISA

- 32-bit fixed format instruction (3 formats)
- Registers
  - 32 32-bit integer GPRs (R1-R31, R0 always = 0)
  - 32 32-bit floating-point GPRs (F0-F31)
    - For double-precision FP, registers paired
- 3-address, reg-reg arithmetic instruction
- Single address mode for load/store: base + displacement
- Simple branch conditions
- Delayed branch

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
$zero	0	The Constant Value 0	N.A.
$at	1	Assembler Temporary	No
$v0-$v1	2-3	Values for Function Results and Expression Evaluation	No
$a0-$a3	4-7	Arguments	No
$t0-$t7	8-15	Temporaries	No
$s0-$s7	16-23	Saved Temporaries	Yes
$t8-$t9	24-25	Temporaries	No
$k0-$k1	26-27	Reserved for OS Kernel	No
$gp	28	Global Pointer	Yes
$sp	29	Stack Pointer	Yes
$fp	30	Frame Pointer	Yes
$ra	31	Return Address	Yes

MIPS Register Use-Convention

- Can write instructions using numbers for registers, from 0 to 31 (e.g. $0, $1, …$31)
- Convention is to use two-character names following a dollar sign to represent a register:
  - $s0, $s1,…: for registers that correspond to variables in C or Java

    `$s0 - $s7` ➔ `$16 - $23`
  - $t0, $t1,…: for temporary registers

    `$t0 - $t7` ➔ `$8 - $15`
  - $a0..$a3: for passing arguments
  - $v0..$v1: values for function results

NAME	NUMBER
$zero	0
$at	1
$v0-$v1	2-3
$a0-$a3	4-7
$t0-$t7	8-15
$s0-$s7	16-23
$t8-$t9	24-25
$k0-$k1	26-27
$gp	28
$sp	29
$fp	30
$ra	31

Convention for Register Names

- To simplify the code in the slides:
  - **Used: R0, R1, R2,….**
  - **Means:$0, $1, $2,….**

- Examples:

```
add    R8,R9,R10
       rd,rs,rt
```

NAME	NUMBER
$zero	0
$at	1
$v0-$v1	2-3
$a0-$a3	4-7
$t0-$t7	8-15
$s0-$s7	16-23
$t8-$t9	24-25
$k0-$k1	26-27
$gp	28
$sp	29
$fp	30
$ra	31

Instruction Formats

- J-format: used for `j` and `jal`
- I-format: used for instructions with immediate nums, `lw` and `sw` (since the offset counts as an immediate), and the branches (`beq` and `bne`),
  - (but not the shift instructions -- later)
- R-format: used for all other instructions
- It will soon become clear why the instructions have been partitioned this way.

MIPS instruction formats

Type	-31-	format (bits)				-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

R-Format Instructions (1/4)

- Define "fields" of the following number of bits each: 6 + 5 + 5 + 5 + 5 + 6 = 32

6	5	5	5	5	6

- For simplicity, each field has a name:

opcode	rs	rt	rd	shamt	funct

- Important: each field is viewed as a 5- or 6-bit unsigned integer,

5-bit fields can represent any number 0-31,
while 6-bit fields can represent any number 0-63

R-Format Instructions (2/4)

- What do these field integer values tell us?
  - `opcode`: partially specifies what the instruction is
    - Note: This number is equal to 0 for all R-Format instructions.
  - `funct`: combined with `opcode`, this number exactly specifies the instruction

Why aren't opcode and funct a single 12-bit field?

R-Format Instructions (3/4)

- More fields:
  - `rs` (Source Register): generally used to specify register containing first operand
  - `rt` (Target Register): generally used to specify register containing second operand

    (note that name is misleading)

  - `rd` (Destination Register): generally used to specify register which will receive result of computation

R-Format Instructions (4/4)

- Final field:
  - **`shamt`**: This field contains the amount a shift instruction will shift by.
    - Shifting a 32-bit word by more than 31 is useless, so this field is only 5 bits (so it can represent the numbers 0-31).
    - This field is set to 0 in all but the shift instructions.

R-Format Example (1/2)

- MIPS Instruction:

```
add    R8,R9,R10
       (rd,rs,rt)
```

`opcode` = 0 (look up in table in book)

`funct` = 32 (look up in table in book)

`rd` = 8 (destination)

`rs` = 9 (first *operand*)

`rt` = 10 (second *operand*)

`shamt` = 0 (not a shift)

R-Format Example (2/2)

- MIPS Instruction:

```
add    R8,R9,R10
```

Decimal number per field representation:

opcode	rs	rt	rd	shamt	funct
0	9	10	8	0	32

Binary number per field representation:

000000	01001	01010	01000	00000	100000

hex

hex representation: 012A 4020$_{hex}$

decimal representation: 19,546,144$_{ten}$

Can R Format Represent All Instructions?

Type	-31-	format (bits)				-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)

What about instructions with immediate values ?

I-Format Instructions (1/4)

- What about instructions with immediate values?
  - 5-bit field only represents numbers up to the value 31: immediates may be much larger than this
  - Ideally, an ISA (e.g. MIPS) would have only one instruction format (for simplicity): unfortunately, **we need to compromise**

Good Design is All About Compromises!

- Define new instruction format that is as consistent as possible (partially consistent) with R-format:
  - First notice that, if instruction has immediate, then it uses at most 2 registers.

Can we Survive with R Format Only?

Type	-31-			format (bits)		-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		

I-Format Instructions (2/4)

- Define "fields" of the following number of bits each: 6 + 5 + 5 + 16 = 32 bits

6	5	5	16

- Again, each field has a name:

opcode	rs	rt	immediate

- Key Concept: Only one field is inconsistent with R-format. Most importantly, `opcode` is still in same location.

R	opcode	rs	rt	rd	shamt	funct
I	opcode	rs	rt	immediate		

I-Format Instructions (3/4)

- **`opcode`**: same as before except that, since there's no **`funct`** field, **`opcode`** uniquely specifies an instruction in I-format
- This is why R-format has two 6-bit fields to identify instruction instead of a single 12-bit field:
  - in order to be as consistent as possible with other formats while leaving as much space as possible for immediate field.
- **`rs`**: specifies a register operand (if there is one)
- **`rt`**: specifies register which will receive result of computation
  - (this is why it's called the **target register** "rt")

I-Format Instructions (4/4)

- The Immediate Field:
  - **`addi`**, **`slti`**, **`sltiu`**, the immediate is sign-extended to 32 bits. Thus, it's treated as a signed integer.
  - 16 bits ➔ can be used to represent immediate up to 2^{16} different values
  - This is large enough to handle the offset in a **typical `lw` or `sw`,** plus a vast majority of values that will be used in the **`slti`** instruction.

I-Format Example (1/2)

- MIPS Instruction:

```
addi    R21,R22,-50
        (rt ,rs ,immediate)
```

`opcode` = 8 (look up in table in book)

`rs` = 22 (register containing operand)

`rt` = 21 (target register)

`immediate` = -50 (by default, this is decimal)

I-Format Example (2/2)

- MIPS Instruction:

```
addi    R21,R22,-50
```

Decimal/field representation:

8	22	21	-50

Binary/field representation:

001000	10110	10101	

hexadecimal representation: 22D5 FFCE$_{hex}$

decimal representation: 584,449,998$_{ten}$

I-Format Problem (1/3)

- Problem:
  - Chances are that **`addi, lw, sw`** and **`slti`** will use immediates small enough to fit in the immediate field (16 bits).

```
Examples:       addi R21,R22,-50
                lw   $t0,12($s0)
                sb   $s0, 3($s1)
                beq   $9,$0,End
```

  - …but what if it's too big?
  - We need a way to deal with a 32-bit *immediate in any I-format* instruction.

I-Format Problem (2/3)

- Solution to Problem:
  - Handle it in software + new instruction
  - Don't change the current instructions: instead, add a new instruction to help out
- New instruction:

```
lui   register, immediate
```

  - stands for **Load Upper Immediate**
  - takes 16-bit immediate and puts these bits in the upper half (high order half) of the specified register
  - sets lower half to 0s

I-Format Problems (3/3)

- Solution to Problem (continued):
  - So how does **`lui`** help us?
  - Example:

```
addi    $t1,$t0, 0xABABCDCD
```

    becomes:

```
lui     $at, 0xABAB
ori     $at, $at, 0xCDCD
add     $t1,$t0,$at
```

  - Now each I-format instruction has only a 16-bit immediate.

Wouldn't it be nice if the assembler do this for us automatically?

Branches: PC-Relative Addressing (1/5)

- Use I-Format

opcode	rs	rt	immediate

  - `opcode` specifies `beq` or `bne`
  - `rs` and `rt` specify registers to compare
  - What can `immediate` specify?
    - `Immediate` is only 16 bits
    - **PC (Program Counter)** has byte address of current instruction being executed; 32-bit pointer to memory
    - So `immediate` cannot specify entire address to branch to.

Branches: PC-Relative Addressing (2/5)

- How do we typically use branches?
  - Answer: `beq $9,$0,End`
  - **`if-else`**, **`while`**, **`for`**
    - Loops are generally small: usually up to 50 instructions
  - Function calls and unconditional jumps are done using jump instructions (`j` and `jal`), not the branches.

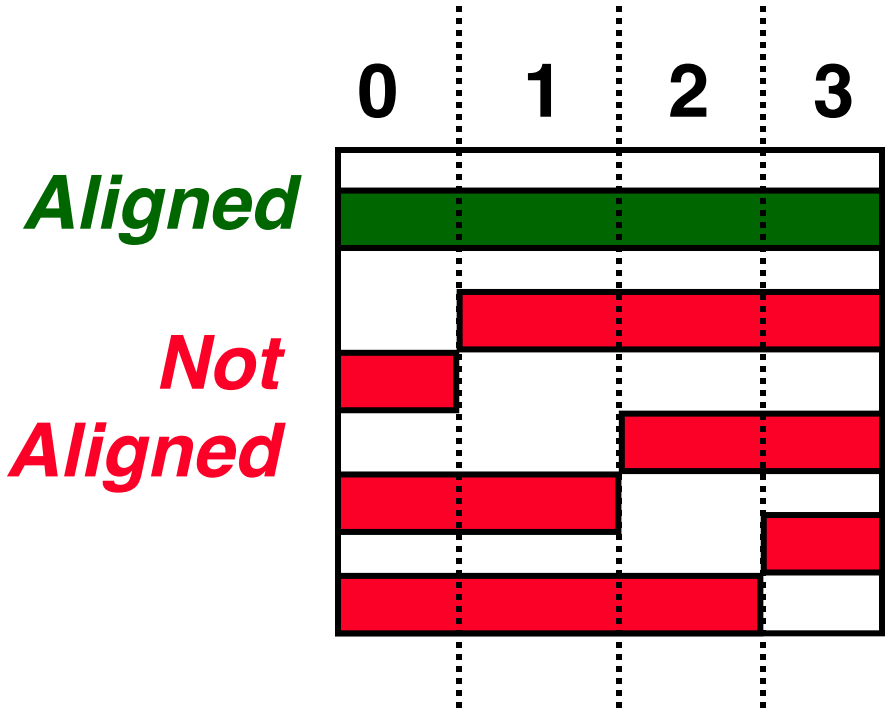
- Conclusion: we may want to branch to anywhere in memory, but a branch often changes **PC** by a small amount

Branches: PC-Relative Addressing (3/5)

- Solution to branches in a 32-bit instruction: **PC-Relative Addressing**
- Let the 16-bit `immediate` field be a signed two's complement integer to be **added** to the PC if we take the branch.
- Now we can branch $\pm 2^{15}$ bytes from the PC, which should be enough to cover almost any loop.
- Any ideas to further optimize this?

More Notes about Memory: Alignment

- MIPS requires that all words in memory start at byte addresses that are multiples of 4 bytes
  - Why? Makes accesses faster



Last digit of address (hex):

0, 4, 8, or C_{hex}

1, 5, 9, or D_{hex}

2, 6, A, or E_{hex}

3, 7, B, or F_{hex}

- Called **Alignment**: objects fall on address that is multiple of their size.

Branches: PC-Relative Addressing (4/5)

- Note: Instructions are words, so they're word aligned (byte address is always a multiple of 4, which means it ends with `00` in binary).
  - So the number of bytes to add to the PC will always be a multiple of 4.
  - So specify the **`immediate`** in **words**.
- Now, we can branch $\pm 2^{15}$ words from the PC (or $\pm 2^{17}$ bytes), so we can handle loops 4 times as large.

Branches: PC-Relative Addressing (5/5)

- Branch Calculation:
  - If we don't take the branch:

    PC = PC + 4

    PC + 4 = byte address of next instruction
  - If we do take the branch:

    PC = (PC + 4) + (**immediate** * 4)
  - Observations
    - **Immediate** field specifies the number of words to jump, which is simply the number of instructions to jump.
    - **Immediate** field can be positive or negative.
    - Due to hardware, add **immediate** to (PC+4), not to PC;

Branch Example (1/3)

- MIPS Code:

```
Loop:   beq   $9,$0,End
        addi  $8,$8,$10
        addi  $9,$9,-1
        j     Loop
End:
```

- **beq** branch is I-Format:
  - **opcode** = 4 (look up in table)
  - **rs** = 9 (first operand)
  - **rt** = 0 (second operand)
  - **immediate** = ???

Branch Example (2/3)

- MIPS Code:

```
Loop:   beq   $9,$0,End
        addi  $8,$8,$10
        addi  $9,$9,-1
        j     Loop
End:
```

- **`Immediate`** Field:
  - Number of instructions to add to (or subtract from) the PC, starting at the instruction *following* the branch.
  - In this case, **`immediate`** = 3

Branch Example (3/3)

- MIPS Code:

```
Loop:   beq   $9,$0,End
        addi  $8,$8,$10
        addi  $9,$9,-1
        j     Loop
End:
```

decimal representation:

4	9	0	3

binary representation:

000100	01001	00000	0000000000000011

Questions on PC-addressing

- Does the value in branch field change if we move the location of code in memory?

No: it is relative to PC

- What do we do if destination is > 2^{15} instructions away from branch?
  - Cannot do it with PC-relative addressing and I-Format

Not much

New instruction format

Can we Survive with R Format Only?

Type	-31-			format (bits)		-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

J-Format Instructions (1/5)

- For branches, we assumed that we don't want to branch too far ➔ we can specify change relative to PC.
- For general jumps (`j` and `jal`) ➔ may jump to anywhere in memory.
- Ideally, we want to specify a 32-bit memory address to jump to.
  - Unfortunately, we can't fit both a 6-bit opcode and a 32-bit address into a single 32-bit word, so we compromise.

J-Format Instructions (2/5)

- Define "fields" of the following number of bits each:

6 bits	26 bits

  - As usual, each field has a name:

opcode	target address

  - Key Concepts
    - Keep **opcode** field identical to R-format and I-format for consistency.
    - Combine all other fields to make room for large target address.

J-Format Instructions (3/5)

- For now, we can specify 26 bits of the 32-bit bit address.
- Optimization:
  - Note that, just like with branches, jumps will only jump to word aligned addresses (instructions), so last two bits are always **00** (in binary).
  - So let's just take this for granted and not even specify them.

J-Format Instructions (4/5)

- Now specify 28 bits of a 32-bit address
- Where do we get the other 4 bits?
  - By definition, take the 4 highest order bits from the PC.
  - Technically, this means that we **cannot** jump to **anywhere** in memory, but it's adequate 99.9999…% of the time, since programs aren't that long
    - only if program needs more than 256 MB
    - Called pseudo-direct or absolute addressing
  - If we absolutely need to specify a 32-bit address, we can always put it in a register and use the **`jr` instruction**.

J-Format Instructions (5/5)

- Summary:
  - New PC = { PC[31..28], target address, 00 }
- Understand where each part came from!
- Note: { , , } means concatenation
  { 4 bits , 26 bits , 2 bits } = 32 bit address
  - { 1010, 11111111111111111111111111, 00 } =
    10101111111111111111111111111100

Summary of MIPS Addressing Modes

- Addressing modes: different ways to specify the address of an operand or instruction
- MIPS has 5 addressing modes:
  1. Immediate
  2. Register
  3. Base or Displacement
  4. PC-Relative
  5. Pseudo-direct or Absolute

```
Examples:       addi R21,R22,-50
                lw   $t0,12($s0)
                sb   $s0, 3($s1)
                beq   $9,$0,End
```

MIPS Addressing Modes (1/2)

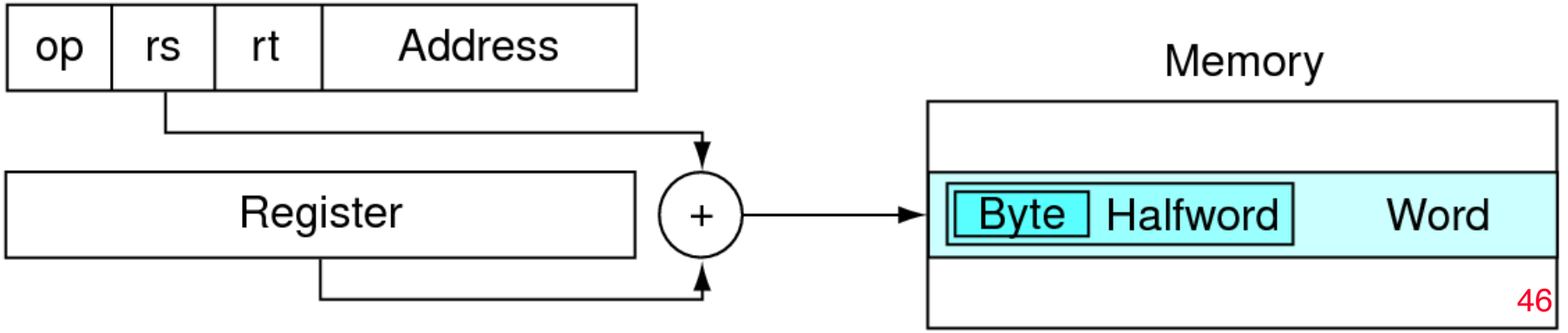
1. Immediate addressing `addi R21,R22,-50`



2. Register addressing `addi R21,R22,R23`

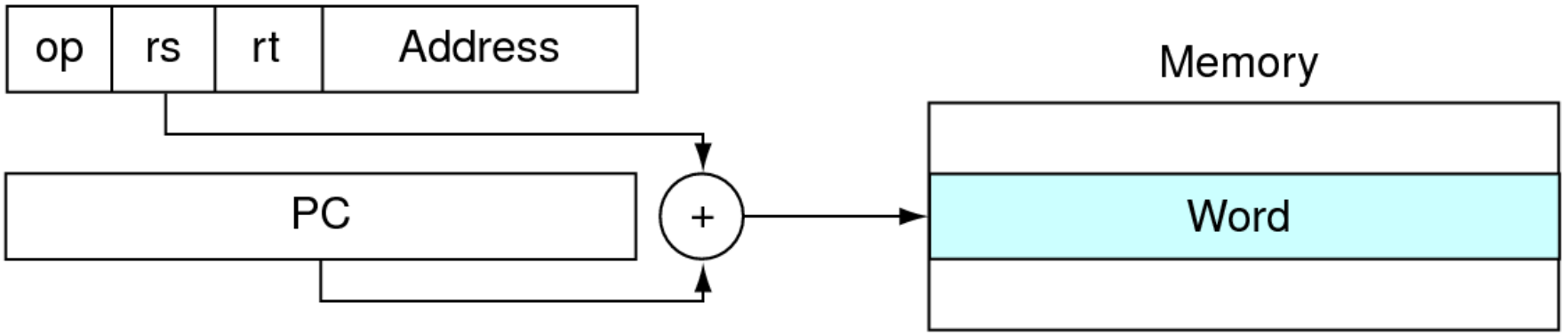


3. Base addressing `lw $t0,12($s0)`

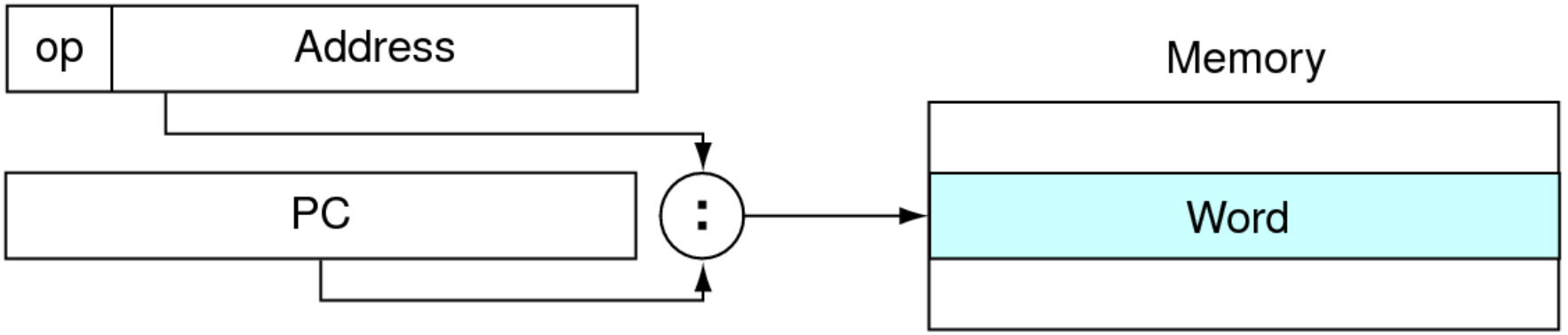


MIPS Addressing Modes (2/2)

4. PC-relative addressing **`beq $9,$0,End`**



5. Pseudodirect addressing jal QuickSort



In conclusion…

- MIPS Machine Language Instruction:
  32 bits representing a single instruction

R	opcode	rs	rt	rd	shamt	funct
I	opcode	rs	rt	immediate		
J	opcode	target address				

- Branches use PC-relative addressing
- Jumps use pseudo-direct (absolute) addressing.

DECODING MACHINE LANGUAGE

Decoding Machine Language

- How do we convert 1s and 0s to assembly language and to C code?

  Machine language ⇒ assembly ⇒ C?

- For each 32 bits:

1. Look at `opcode` to distinguish between R-Format, J-Format, and I-Format.

R	0
I	1, 4–31
J	2 or 3

2. Use instruction format to determine which fields exist.
3. Write out MIPS assembly code, converting each field to name, register number/name, or decimal/hex number.
4. Logically convert this MIPS code into valid C code. Always possible? Unique?

Decoding Example (1/7)

- Here are six machine language instructions in hexadecimal:

00001025_{hex}
$0005402A_{hex}$
11000003_{hex}
00441020_{hex}
$20A5FFFF_{hex}$
08100001_{hex}

- Let the first instruction be at address $4{,}194{,}304_{ten}$ ($0x00400000_{hex}$).
- Next step: convert hex to binary

Decoding Example (2/7)

- The six machine language instructions in binary:

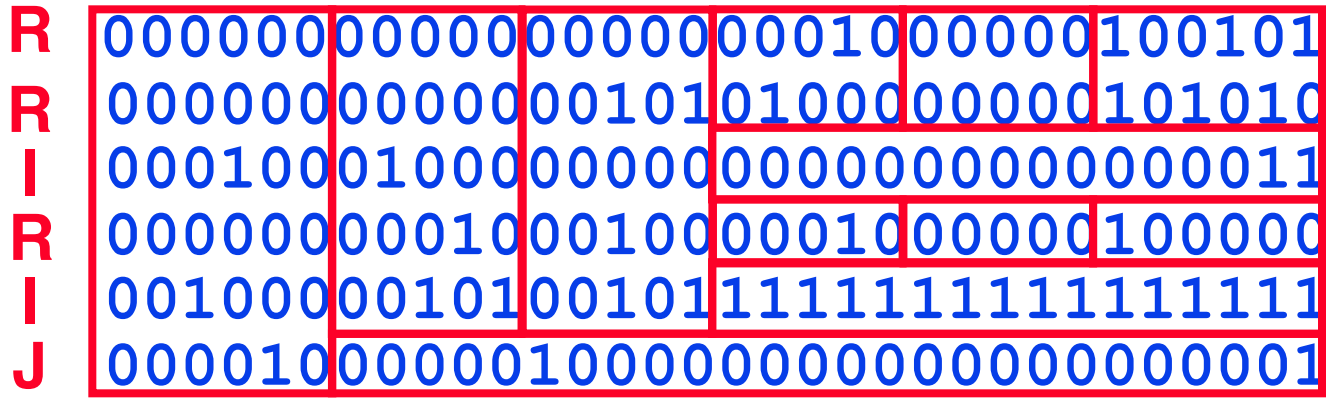
```
00000000000000000001000000100101
00000000000001010100000000101010
00010001000000000000000000000011
00000000010001000001000000100000
00100000101001011111111111111111
00001000000100000000000000000001
```

Next step: identify opcode and format

R	0	rs	rt	rd	shamt	funct
I	1, 4-31	rs	rt	immediate		
J	2 or 3	target address				

Decoding Example (3/7)

- Select the opcode (first 6 bits) to determine the format:



- Look at **opcode**:
  0 means R-Format,
  2 or 3 mean J-Format,
  otherwise I-Format.
- Next step: separation of fields

Decoding Example (4/7)

- Fields separated based on format/opcode:

Format:

R	000000	00000	00000	00010	00000	100101
R	000000	00000	00101	01000	00000	101010
I	000100	01000	00000	0000000000000011		
R	000000	00010	00100	00010	00000	100000
I	001000	00101	00101	1111111111111111		
J	000010	00000100000000000000000001				

- Next step: translate ("disassemble") to MIPS assembly instructions

Decoding Example (4/7)

- Fields separated based on format/opcode:

Format:

R	0	0	0	2	0	37
R	0	0	5	8	0	42
I	4	8	0	+3		
R	0	2	4	2	0	32
I	8	5	5	-1		
J	2	1,048,577				

- Next step: translate ("disassemble") to MIPS assembly instructions

Decoding Example (5/7)

- MIPS Assembly (Part 1):

Address	Assembly instructions
0x00400000	or $2,$0,$0
0x00400004	slt $8,$0,$5
0x00400008	beq $8,$0,3
0x0040000c	add $2,$2,$4
0x00400010	addi $5,$5,-1
0x00400014	j 0x100001

NAME	NUMBER
$zero	0
$at	1
$v0-$v1	2-3
$a0-$a3	4-7
$t0-$t7	8-15
$s0-$s7	16-23
$t8-$t9	24-25
$k0-$k1	26-27
$gp	28
$sp	29
$fp	30
$ra	31

- Better solution: translate to more meaningful MIPS instructions (fix the branch/jump and add labels, registers)

Decoding Example (6/7)

- MIPS Assembly (Part 2):

```
        or    $v0,$0,$0
Loop:   slt   $t0,$0,$a1
        beq   $t0,$0,Exit
        add   $v0,$v0,$a0
        addi  $a1,$a1,-1
        j     Loop
Exit:
```

NAME	NUMBER
$zero	0
$at	1
$v0-$v1	2-3
$a0-$a3	4-7
$t0-$t7	8-15
$s0-$s7	16-23
$t8-$t9	24-25
$k0-$k1	26-27
$gp	28
$sp	29
$fp	30
$ra	31

- Next step: translate to C code (must be creative!)

MIPS Register Convention

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
$zero	0	The Constant Value 0	N.A.
$at	1	Assembler Temporary	No
$v0-$v1	2-3	Values for Function Results and Expression Evaluation	No
$a0-$a3	4-7	Arguments	No
$t0-$t7	8-15	Temporaries	No
$s0-$s7	16-23	Saved Temporaries	Yes
$t8-$t9	24-25	Temporaries	No
$k0-$k1	26-27	Reserved for OS Kernel	No
$gp	28	Global Pointer	Yes
$sp	29	Stack Pointer	Yes
$fp	30	Frame Pointer	Yes
$ra	31	Return Address	Yes

Decoding Example (7/7)

Before Hex:

00001025_{hex}
$0005402A_{hex}$
11000003_{hex}
00441020_{hex}
$20A5FFFF_{hex}$
08100001_{hex}

- **After C code (Mapping below)**

$v0: product
$a0: multiplicand
$a1: multiplier

```
product = 0;
  while (multiplier > 0) {
     product += multiplicand;
     multiplier -= 1;
  }
```

```
        or   $v0,$0,$0
Loop:   slt  $t0,$0,$a1
        beq  $t0,$0,Exit
        add  $v0,$v0,$a0
        addi $a1,$a1,-1
        j    Loop
Exit:
```

Big Idea:

Instructions are just numbers, code is treated like data

PSEUDO INSTRUCTION

Recall lui?

- So how does **`lui`** help us?
- Example:

```
addi    $t0,$t0, 0xABABCDCD
```

becomes:

```
lui     $at, 0xABAB
ori     $at, $at, 0xCDCD
add     $t0,$t0,$at
```

- Now each I-format instruction has only a 16-bit immediate.
- Wouldn't it be nice if the assembler would this for us automatically?

True Assembly Language (1/3)

- **Pseudo Instruction**: A MIPS instruction that doesn't turn directly into a machine language instruction, but into other MIPS instructions
- What happens with pseudo-instructions?
  - They're broken up by the assembler into several "real" MIPS instructions
- The purpose of pseudo Instructions is to make writing assembly code easier

Example Pseudo Instructions

- Register Move

```
move   reg2,reg1
```

Expands to:

```
add    reg2,$zero,reg1
```

- Load Immediate

```
li     reg,value
```

If value fits in 16 bits:

```
addi   reg,$zero,value
```

else:

```
lui    reg, {upper 16 bits}
ori    reg, reg,{lower 16 bits}
```

Example Pseudo instructions

- Load Address: How do we get the address of an instruction or global variable into a register?

```
la          reg, label
```

Again if value fits in 16 bits:

```
addi    reg,$zero,label_value
```

else:

```
lui     reg, {upper 16 bits of value}
ori     reg, reg, {lower 16 bits}
```

True Assembly Language (2/3)

- Problem:
  - When breaking up a pseudo-instruction, the assembler may need to use an extra reg.
  - If it uses any regular register, it'll overwrite whatever the program has put into it.

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
$zero	0	The Constant Value 0	N.A.
$at	1	Assembler Temporary	No
$v0-$v1	2-3	Values for Function Results and Expression Evaluation	No
$a0-$a3	4-7	Arguments	No
$t0-$t7	8-15	Temporaries	No
$s0-$s7	16-23	Saved Temporaries	Yes
$t8-$t9	24-25	Temporaries	No
$k0-$k1	26-27	Reserved for OS Kernel	No
$gp	28	Global Pointer	Yes
$sp	29	Stack Pointer	Yes
$fp	30	Frame Pointer	Yes
$ra	31	Return Address	Yes

- Solution:
  - Reserve a register (`$1`, called `$at` for "assembler temporary") that assembler will use to break up pseudo-instructions.
  - Since the assembler may use this at any time, it's not safe to code with it.

Example Pseudo Instructions

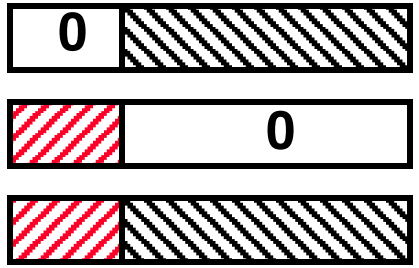
- Rotate Right Instruction

```
ror     reg, value
```



Expands to:

```
srl     $at, reg, value
sll     reg, reg, 32-value
or      reg, reg, $at
```



- "No OPeration" instruction (does nothing):

```
nop
```

Expands to instruction =

```
sll     $0, $0, 0
```

Example Pseudo Instructions

- Wrong operation for operand

```
addu   reg,reg,value # should be addiu
```

If value fits in 16 bits, `addu` is changed to:

```
addiu  reg,reg,value
```

else:

```
lui    $at,upper 16 bits of value
ori    $at,$at,lower 16 bits
addu   reg,reg,$at
```

- How do we avoid confusion about whether we are talking about MIPS assembler with or without pseudo instructions?

True Assembly Language (3/3)

- MAL (MIPS Assembly Language): the set of instructions that a programmer may use to code in MIPS; this **includes pseudo-instructions**
- TAL (True Assembly Language): set of instructions that can actually get translated into a single machine language instruction (32-bit binary string)
- A program must be converted from MAL into TAL before translation into 1s & 0s.

Questions on Pseudoinstructions

- Question:
  - How does MIPS assembler recognize pseudo-instructions?
- Answer:
  - It looks for officially defined pseudo-instructions, such as **`ror`** and **`move`**
  - It looks for special cases where the operand is incorrect for the operation and tries to handle it gracefully

Rewrite TAL as MAL

- TAL:

```
            or    $v0,$0,$0
Loop:       slt   $t0,$0,$a1
            beq   $t0,$0,Exit
            add   $v0,$v0,$a0
            addi  $a1,$a1,-1
            j     Loop
Exit:
```

- This time convert to MAL
- It's OK for this exercise to make up MAL instructions

Rewrite TAL as MAL (Answer)

- **TAL:**

```
        or    $v0,$0,$0
Loop:   slt   $t0,$0,$a1
        beq   $t0,$0,Exit
        add   $v0,$v0,$a0
        addi  $a1,$a1,-1
        j     Loop
Exit:
```

- **MAL:**

```
        li    $v0,0
Loop:   ble   $a1,$zero,Exit
        add   $v0,$v0,$a0
        sub   $a1,$a1,1
        j     Loop
Exit:
```

Lec_03 (Encryption and Security)

C TO MIPS ASSEMBLY EXAMPLES

Examples:

A single line of C may break up into several lines of MIPS

```
a = b + c (in C)
add  $s0,$s1,$s2 (in MIPS)
```

```
d = e - f (in C)
sub  $s3,$s4,$s5 (in MIPS)
```

Examples:

```
a = b + c + d - e;

add $t0, $s1, $s2   # temp = b + c
add $t0, $t0, $s3   # temp = temp + d
sub $s0, $t0, $s4   # a = temp - e
```

```
f = (g + h) - (i + j);

add $t0,$s1,$s2     # temp = g + h
add $t1,$s3,$s4     # temp = i + j
sub $s0,$t0,$t1     # f=(g+h)-(i+j)
```

Examples:

- `f = g` (in C)
- `add $s0,$s1,$zero` (in MIPS)

- `add $zero,$zero,$s0` (in MIPS)
- will not do anything!

- `f = g + 10` (in C)
- `addi $s0,$s1,10` (in MIPS)
- `f = g - 10` (in C)
- `addi $s0,$s1,-10` (in MIPS)
- ~~No subi in MIPS~~

Data Transfer between Mem and Reg

- Remember: "Load FROM memory"
- Remember: "Store INTO memory"

- To specify a memory address to copy from, specify two things:
  - A base register containing a pointer to memory
  - A numerical offset (in bytes)
  - The desired memory address is the sum of these two values.
- Example: `8($t0)`
  - memory address pointed to by the value in `$t0`, plus 8 bytes

Data Transfer between Mem and Reg

- Remember: "Load FROM memory"



- Example:
```
lw $t0,12($s0)
lb $s0, 3($s1)
```

- Remember: "Store INTO memory"



- Example:
```
sw $t0,12($s0)
sb $s0, 3($s1)
```

What about the other 24 bits in the 32 bit register?

Arrays

- **Memory[0], Memory[1], Memory[2], …**

Called the "**address**" of a word

- **A** is an array in C? (**A** is **int** array)
- **A[5]** (5th int in **A**)

```
g = h + A[5];  #g: $s1, h: $s2, $s3: base address of A
```

```
lw $t0,20($s3)     # $t0 gets A[5]
add $s1,$s2,$t0    # $s1 = h + A[5]
```

C Decisions: if Statements

- 2 kinds of `if` statements in C

  `if` (*condition*) *clause*

  `if` (*condition*) *clause1* **`else`** *clause2*

- Rearrange 2nd `if` into following:

```
if   (condition) goto L1;
     clause2;
     goto L2;
L1:  clause1;
L2:
```

- **`L1`** and **`L2`** are called **labels** (used in case statements)
- **`goto`** is **unstructured**
  - Not as elegant as `if-else`, but same meaning
  - **Horrible C programming style**
  - Necessary in MIPS Assembly!

MIPS Decision Instructions

- Decision instruction in MIPS:

```
beq   register1, register2, L1
```

beq is "Branch if (registers are) equal"

Same meaning as (using C):

```
if  (register1==register2) goto L1
```

- Complementary MIPS decision instruction

```
bne   register1, register2, L1
```

bne is "Branch if (registers are) not equal"

Same meaning as (using C):

```
if  (register1!=register2) goto L1
```

- Called conditional branches

MIPS Goto Instruction

- In addition to conditional branches, MIPS has an unconditional branch:

```
j   label
```

- Called a Jump Instruction: jump (or branch) directly to the given label without needing to satisfy any condition
- Same meaning as (using C):

```
goto label
```

- Technically, it's the same effect as:

```
beq $0,$0,label
```

since it always satisfies the condition.

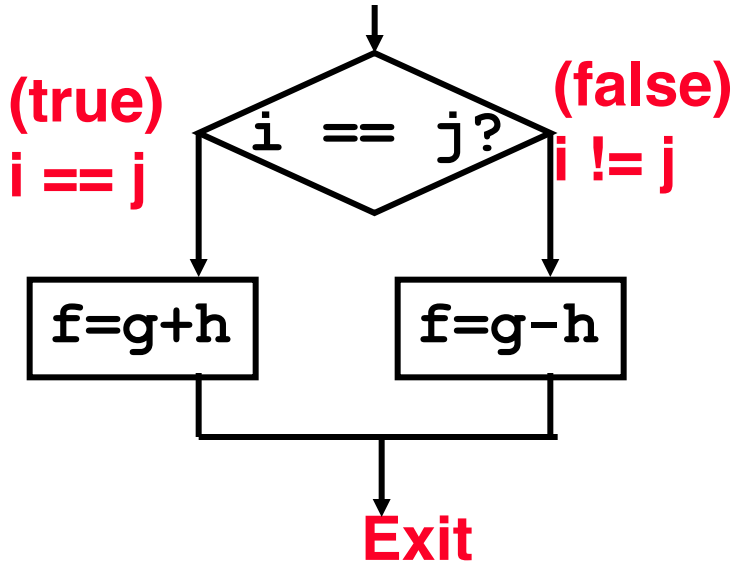
Compiling C if into MIPS (1/2)

- Compile by hand

```
if (i == j) f=g+h;
else f=g-h;
```

- Use this mapping:

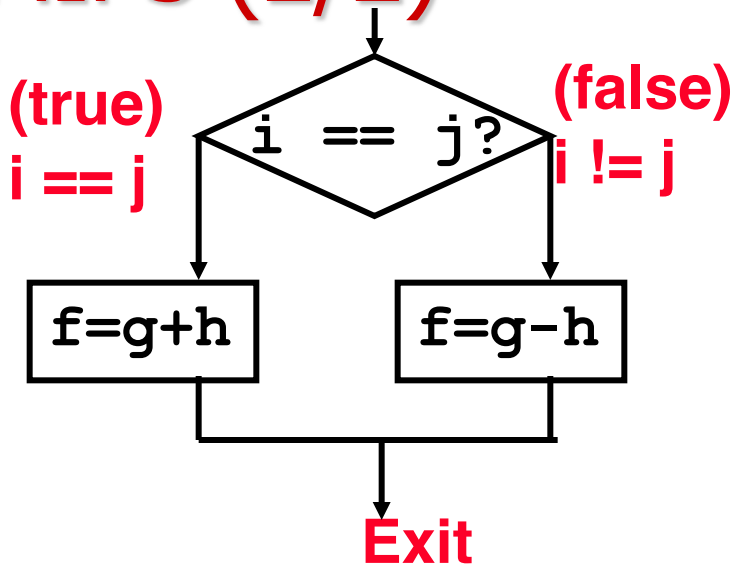
```
f: $s0
g: $s1
h: $s2
i: $s3
j: $s4
```



Compiling C if into MIPS (2/2)

- **Compile by hand**

```
if (i == j) f=g+h;
else f=g-h;
```



- Final compiled MIPS code:

```
      beq $s3,$s4,True     # branch i==j
       sub $s0,$s1,$s2     # f=g-h(false)
       j   Fin             # goto Fin
True: add $s0,$s1,$s2      # f=g+h (true)
Fin:
```

Note: Compiler automatically creates labels to handle decisions (branches). Generally not found in HLL code.

Loops in C/Assembly (1/3)

- Simple loop in C; `A[]` is an array of `int(s)`

```
do {
        g = g + A[i];
        i = i + j;
} while (i != h);
```

- Rewrite this as:

```
Loop: g = g + A[i];
      i = i + j;
      if (i != h) goto Loop;
```

- Use this mapping:

```
  g,   h,   i,   j, base of A
$s1, $s2, $s3, $s4, $s5
```

Loops in C/Assembly (2/3)

- Final compiled MIPS code:

```
Loop:                          #$t1= 4*i
                               #$t1=addr A[i]
                              #$t1=A[i]
                              #g=g+A[i]
                              #i=i+j
                              # goto Loop
                              # if i!=h
```

- Original code:

```
Loop:    g = g + A[i];
         i = i + j;
         if (i != h) goto Loop;

        g,   h,   i,   j, base of A
      $s1, $s2, $s3, $s4, $s5
```

Loops in C/Assembly (2/3)

- Final compiled MIPS code:

```
Loop: sll $t1,$s3,2      #$t1= 4*i
      add $t1,$t1,$s5    #$t1=addr A[i]
      lw  $t1,0($t1)     #$t1=A[i]
      add $s1,$s1,$t1    #g=g+A[i]
      add $s3,$s3,$s4    #i=i+j
      bne $s3,$s2,Loop   # goto Loop
                         # if i!=h
```

- Original code:

```
Loop:   g = g + A[i];
        i = i + j;
        if (i != h) goto Loop;
```

g, h, i, j, base of A
$s1, $s2, $s3, $s4, $s5

Loops in C/Assembly (3/3)

- There are three types of loops in C:
  - `while`
  - `do… while`
  - `for`
- Each can be rewritten as either of the other two, so the method used in the previous example can be applied to `while` and `for` loops as well.
- Key Concept: Though there are multiple ways of writing a loop in MIPS, the key to decision making is conditional branch

Inequalities in MIPS (1/2)

- Until now, we've only tested equalities (== and !=)
  General programs need to test < and > as well.
  - "Set on Less Than"
  - Syntax: `slt reg1,reg2,reg3`
  - Meaning:

```
OR:     if (reg2 < reg3)
                reg1 = 1;     // set reg1
        else reg1 = 0;        // reset reg1
```

"set" means "set to 1"
"reset" means "set to 0"

Inequalities in MIPS (2/2)

- How do we use this? Compile by hand:

```
if (g < h) goto Less; #g:$s0,h:$s1
```

- Compiled MIPS code…

```
slt $t0,$s0,$s1 # $t0 = 1 if g<h
bne $t0,$0,Less # goto Less
                # if $t0!=0
                # (if (g<h)) Less:
```

- Register `$0` always contains the value `0`, so `bne` and `beq` often use it for comparison after an `slt` instruction.
- `slt` → `bne` pair means `if(... < ...)goto...`

Logical Operators

- Instruction Names:
  - `and, or`: Both of these expect the third argument to be a register
  - `andi, ori`: Both of these expect the third argument to be an immediate
- MIPS Logical Operators are all bitwise, meaning that bit 0 of the output is produced by the respective bit 0's of the inputs, bit 1 by the bit 1's, etc.
  - C: Bitwise AND is `&` (e.g., `z = x & y;`)
  - C: Bitwise OR is `|` (e.g., `z = x | y;`)
- MIPS also has `xor, xori, nor` instructions

Uses for Logical Operators (1/3)

- Note that **and**ing a bit with 0 produces a 0 at the output while **and**ing a bit with 1 produces the original bit.
- This can be used to create a mask.
  - Example:

```
        1011 0110 1010 0100 0011 1101 1001 1010
mask:   0000 0000 0000 0000 0000 1111 1111 1111
```

  - The result of **and**ing these:

```
        0000 0000 0000 0000 0000 1101 1001 1010
```

mask most significant 20 bits

Uses for Logical Operators (2/3)

- The second bitstring in the example is called a mask. It is used to isolate the rightmost 12 bits of the first bitstring by masking out the rest of the string (e.g. setting it to all 0s).
- Thus, the **`and`** operator can be used to set certain portions of a bitstring to 0s, while leaving the rest alone.
  - In particular, if the first bitstring in the above example were in **`$t0`**, then the following instruction would mask it:

```
andi    $t0,$t0, 0xFFF
```

Uses for Logical Operators (3/3)

- Similarly, note that oring a bit with 1 produces a 1 at the output while oring a bit with 0 produces the original bit.
- This can be used to force certain bits of a string to 1s.
  - For example, if `$t0` contains `0x12345678`, then after this instruction:

    ```
    ori  $t0, $t0, 0xFFFF
    ```

    `$t0` contains `0x1234FFFF` (e.g. the high-order 16 bits are untouched, while the low-order 16 bits are forced to 1s).

MIPS example

- Assume (initially)
  - R1 = 16, R2 = 0, R3 = 4, R4 = 19, R5 = 44 *(all decimal values)*
  - Mem[0] = 0x12345678
  - Mem[4] = 0xAABBCCDD
  - Mem[8] = 0x99663300
  - Mem[12] = 0xDEADBEEF
  - Assume memory is big-endian
- For each instruction, list:
  - What register, if any, is changed
  - What memory location, if any, is changed
  - The new values of any changed register/memory

```
I0:     DADD R1, R2, R3
I1:     DSUB R4, R5, R1
I2:     LB   R3, 0(R1)
I3:     SLT  R5, R3, R1
I4:     BEQ  R5, R0, end
I5:     SB   R4, 0(R1)
end:    SH   R4, 4(R1)
```

MIPS example solution

- `I0: DADD R1, R2, R3`
  - R1 = R2 + R3 = 0 + 4 = **4**
- `I1: DSUB R4, R5, R1`
  - R4 = R5 – R1 = 44 – 4 = **40**
- `I2: LB R3, 0(R1)`
  - R3 = mem[0+[R1]] = mem[0+4] = mem[4]
  - LB → load single byte and sign-extend
  - Therefore, R3 = **0xFFFFFFAA**
- `I3: SLT R5, R3, R1`
  - R5 = 1 if R3 < R1 → 0xFFFFFFAA < 4
  - MSb of R3 is 1 → 0xFFFFFFAA is negative (-86)
  - Therefore, R3 < R1 → R5 = **1**

MIPS example solution (cont.)

- `I4: BEQ  R5, R0, end`
  - Branch to label "end" if (R5 == R0)
    - Remember, R0 always equals 0
  - R5 = 1, R0 = 0 → **do not branch**
  - Note: if R5 == R0, next instruction skipped
- `I5: SB  R4, 0(R1)`
  - Write byte from R4 to mem[0+[R1]] → mem[4]
    - R4 = 40 = 0x00000028
    - SB, SH take least-significant byte(s) from register
  - Showed whole word initially; only 1 byte changes
    - mem[4] = 0x**28**BBCCDD
- `end:SH  R4, 4(R1)`
  - Write half-word (2 bytes) from R4 to mem[8]
    - mem[8] = 0x**0028**3300

Next time

- Implementing an ISA